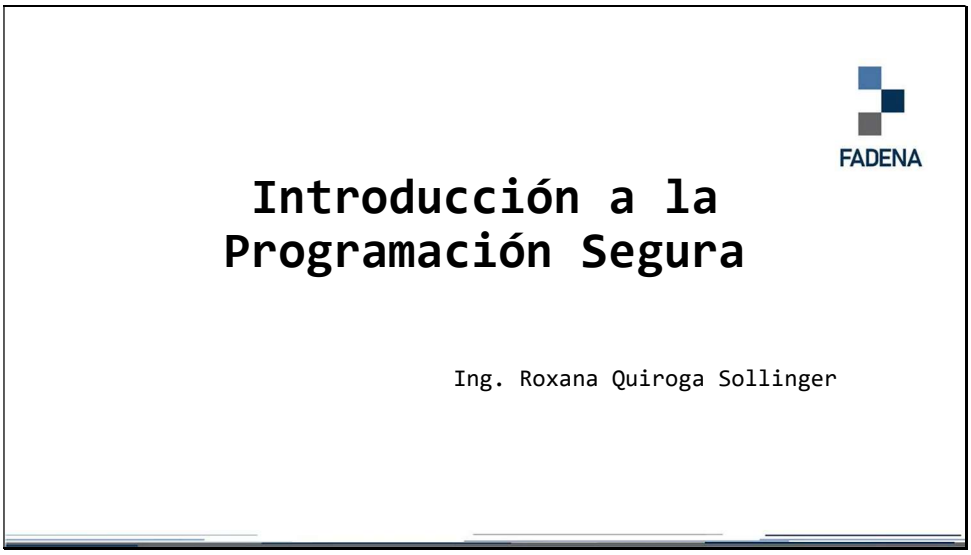
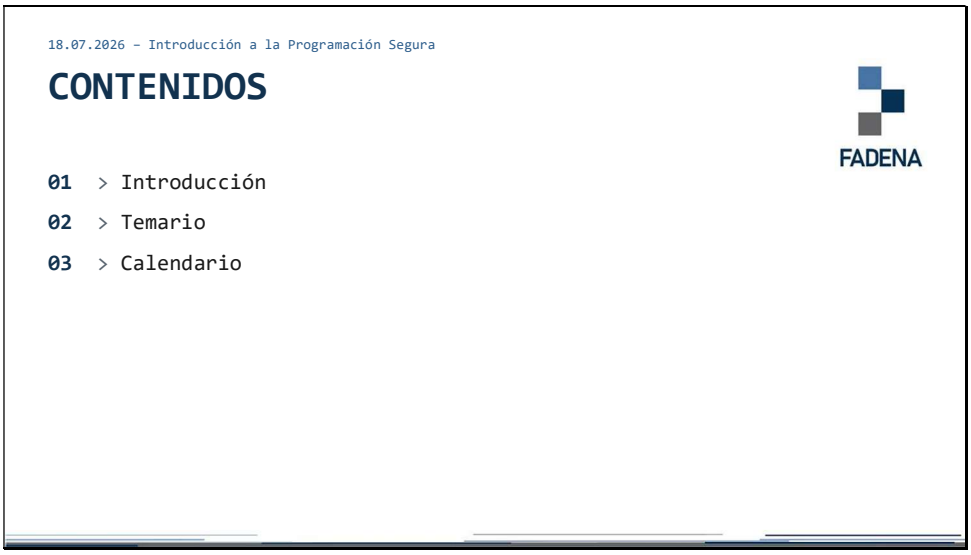
Slide 1



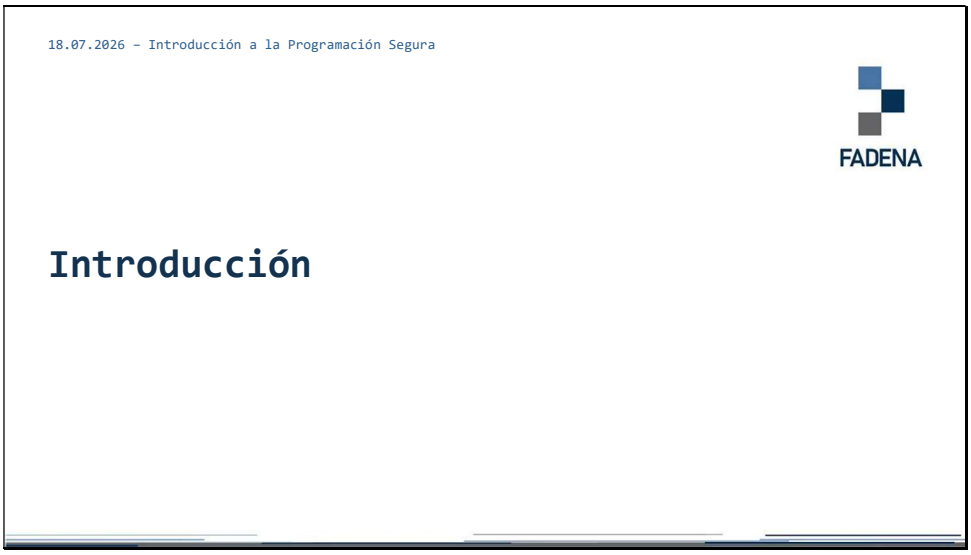
¡Hola y bienvenidos a Programación Segura! Soy la Ing. Roxana Quiroga Sollinger. En esta introducción repasaremos los objetivos principales de la asignatura, sus contenidos y el cronograma del semestre.

Slide 2



Para comenzar, revisaremos la materia en tres bloques: Primero, una introducción al curso y a sus objetivos principales. Segundo, el temario con el detalle de cada contenido. Y por último, el calendario del cuatrimestre —de agosto a diciembre—, donde verán el avance semana a semana y los períodos previstos para las evaluaciones. Aclaración importante: señalar la semana de un parcial o final no significa que el examen vaya a estar abierto los siete días, sino que se rendirá en un día específico dentro de esa fecha.

Slide 3



Introducción

Slide 4



Hoy nos adentraremos en los objetivos de la cátedra de la **Programación Segura**, un pilar crítico en el desarrollo de software moderno.
A lo largo de esta session introductoria abordaremos tres ejes principales: en primer lugar, cómo integrar la seguridad en el **Ciclo de Vida del Software**; en segundo lugar, las **metodologías existentes** para analizar riesgos y modelar amenazas; y finalmente, las **técnicas de mitigación** esenciales para prevenir y corregir vulnerabilidades.

Comencemos con el primer punto: los **Lineamientos para el Ciclo de Vida de Desarrollo del Software** (o SSDLC por sus siglas en inglés).
Históricamente, la seguridad se trataba como una auditoría final, justo antes de lanzar un producto a producción. Hoy sabemos que ese enfoque es ineficiente y costoso. Nuestro objetivo central aquí es aplicar el principio de **'Shift Left'** o 'Desplazar a la izquierda': mover las prácticas de seguridad hacia las etapas más tempranas del desarrollo, como la planificación y el diseño.
Además, para que esto sea sostenible en entornos ágiles o DevSecOps, la seguridad no puede ser un freno; debe integrarse como un **proceso continuo y automatizado** mediante pruebas estáticas y dinámicas dentro del flujo de integración continua.

Pasando al segundo bloque, nos encontramos con las **Metodologías de Análisis, Modelado de Amenazas y Riesgo**.
No podemos proteger lo que no entendemos. Para identificar vulnerabilidades antes de que afecten a nuestros usuarios, debemos estructurar metodologías específicas según el nivel de abstracción del sistema.
Esto significa analizar desde la arquitectura general —identificando qué actores interactúan con el sistema y por dónde viajan los datos sensibles— hasta el nivel de código. El **modelado de amenazas** nos permite preguntarnos de forma sistemática:

¿Cuáles son los riesgos?, *¿Cómo pueden atacar este componente?* y *¿Qué haremos para evitarlo?*, permitiéndonos priorizar los riesgos reales sobre las suposiciones.

Por último, llegamos al tercer bloque: las **Técnicas y Controles de Mitigación**. Una vez identificados los riesgos, debemos construir las defensas.
Aquí el principio rector es la **Defensa en Profundidad**. Este concepto establece que nunca debemos confiar en una sola medida de seguridad. Si un atacante logra superar una barrera, debe encontrarse inmediatamente con otra.
Aplicar múltiples capas implica combinar validación de entradas en el código, autenticación robusta, cifrado de datos tanto en tránsito como en reposo, y configuraciones seguras en la infraestructura. Si una capa falla, las demás protegen el activo.

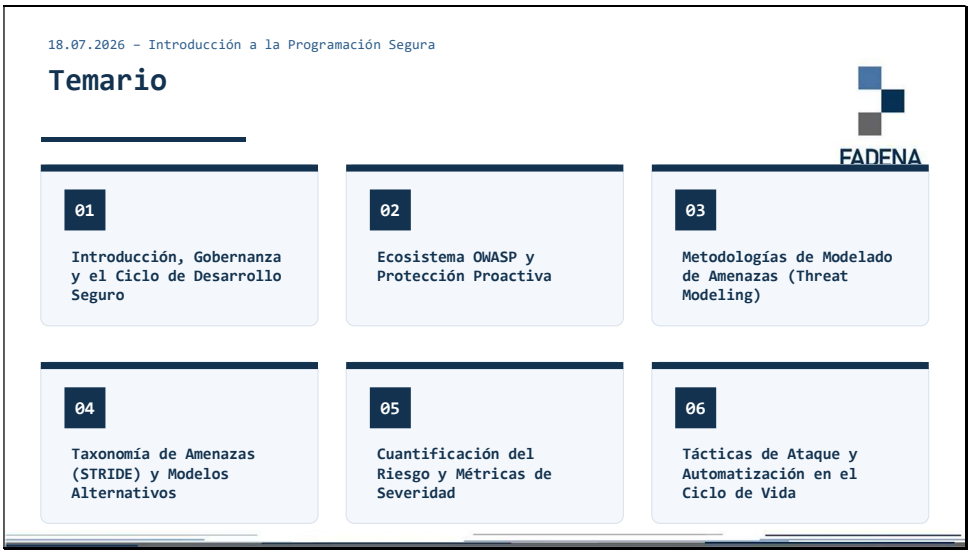
En resumen, la programación segura no es un parche que se añade al final, sino una disciplina que se cultiva desde el primer requerimiento hasta el despliegue final. Adoptar estos tres pilares es la clave para construir software resiliente, confiable y preparado para las amenazas del mundo real.

Slide 5



Diapositiva de transición: cerramos la introducción y pasamos al temario de la materia.
Anticipar que el temario está organizado en seis bloques progresivos, que van desde los fundamentos y la gobernanza hasta las tácticas de ataque y la automatización en el pipeline.
Aclarar que cada bloque se dicta en semanas consecutivas y que el calendario posterior muestra las fechas exactas.

Slide 6



A lo largo del curso, recorreremos un camino estructurado en 6 bloques clave para transformar la seguridad en una pieza fundamental del desarrollo:

Bloque 1: Introducción, Gobernanza y el Ciclo de Desarrollo Seguro
Arrancamos entendiendo el impacto económico de la inseguridad y la cultura DevSecOps bajo los pilares CAMS. Adoptaremos la arquitectura *Zero Trust* y APIs, y analizaremos las fases obligatorias del modelo MSSDL de Microsoft, desde la definición de requerimientos hasta la respuesta ante incidentes.

Bloque 2: Ecosistema OWASP y Protección Proactiva
Nos sumergimos en la comunidad OWASP para implementar una estrategia de defensa en profundidad. Aprenderás a validar entradas, codificar salidas y gestionar datos sensibles, además de abordar la gestión de identidades, autenticación y criptografía orientada a código seguro.

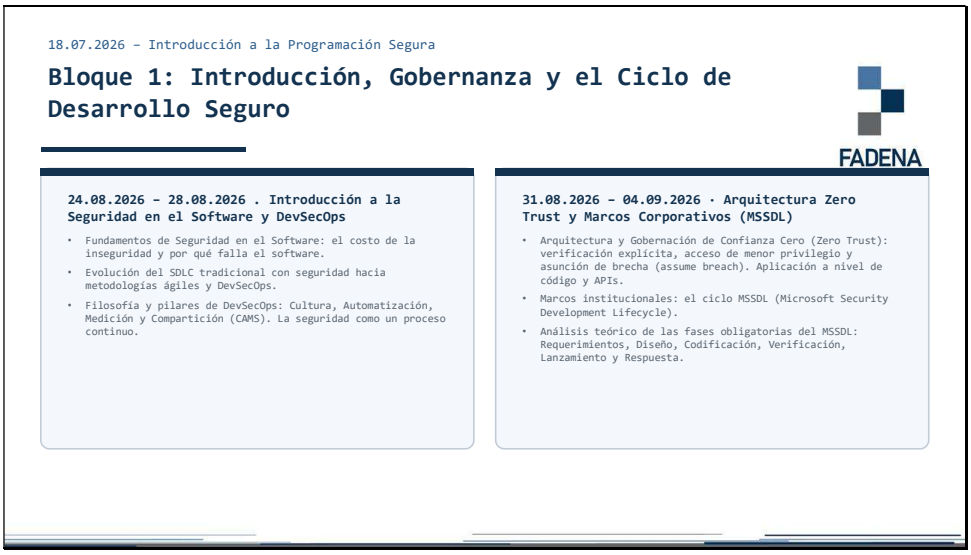
Bloque 3: Metodologías de Modelado de Amenazas (Threat Modeling)
Descubriremos cómo anticiparnos a los ataques antes de escribir código. Compararemos modelos de aplicación versus infraestructura, usaremos la metodología ágil **VAST** y recorreremos paso a paso las 7 fases del marco **PASTA**, alineando los objetivos de negocio con el análisis técnico de riesgos e impacto.

Bloque 4: Taxonomía de Amenazas (STRIDE) y Modelos Alternativos
Desglosaremos en detalle el modelo **STRIDE** para identificar vulnerabilidades como la suplantación, alteración de datos, repudio o elevación de privilegios. Complementaremos esto con metodologías orientadas a activos como **TRIKE** y la creación de diagramas de flujo de datos seguros.

Bloque 5: Cuantificación del Riesgo y Métricas de Severidad
Aprenderás a priorizar y medir el riesgo real de las vulnerabilidades. Dominaremos el nuevo estándar **CVSS v4.0** —analizando sus métricas base, ambientales y suplementarias— junto a metodologías cualitativas e híbridas como **DREAD**, **OCTAVE** y **QTMM**.

Bloque 6: Tácticas de Ataque y Automatización en DevSecOps
Cerramos analizando el comportamiento real del atacante mediante la matriz **MITRE ATT&CK**. Llevaremos todo esto a la práctica automatizando análisis SAST, DAST y SCA directamente en pipelines de CI/CD, fomentando una cultura de gobernanza con la figura del *Security Champion*.

Slide 7



En esta etapa inicial nos enfocaremos en sentar las bases teóricas, la arquitectura y los marcos de gobernanza que permiten transformar la seguridad de un software: dejando de verla como un parche de último momento para convertirla en la columna vertebral del proceso de desarrollo.

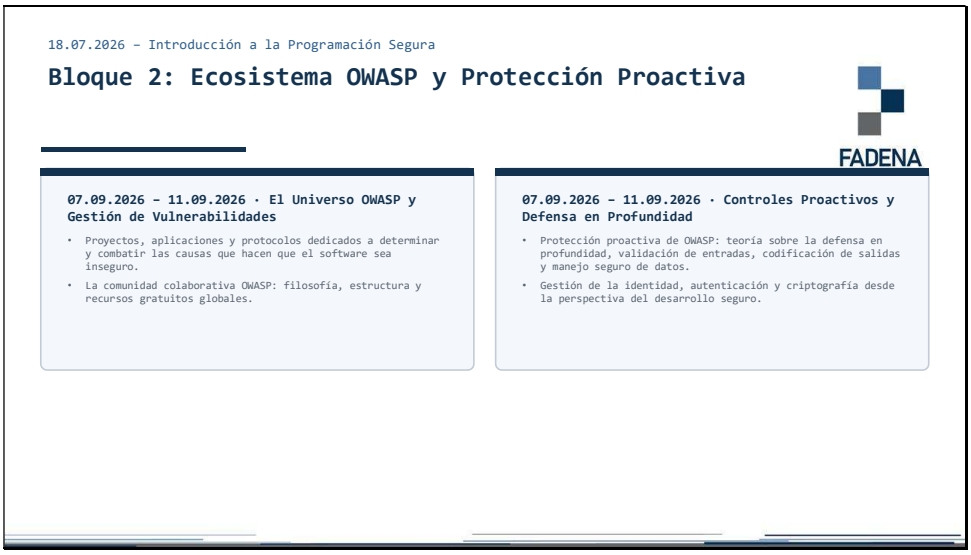
Durante la segunda semana del 24 de Agosto hasta el 28 de Agosto comenzaremos abordando una realidad crítica: ¿por qué falla el software y cuál es el impacto económico real de la inseguridad? Históricamente, la seguridad se trataba como una fase aislada al final del proyecto, una especie de "peaje" justo antes de salir a producción. Sin embargo, con el ritmo acelerado de las metodologías ágiles, este modelo tradicional colapsó.
Evolucionaremos hacia el paradigma **DevSecOps**, comprendiendo que no se trata solo de implementar herramientas automáticas, sino de adoptar una filosofía de trabajo continua. Desglosaremos los cuatro pilares del modelo **CAMS**: la **Cultura** de responsabilidad compartida, la **Automatización** en los procesos, la **Medición** constante de métricas y la **Compartición** del conocimiento entre los equipos de desarrollo, operaciones y seguridad.

Con estas bases claras, en la segunda mitad del Bloque del 31 de Agosto hasta el 4 de Septiembre nos adentraremos en el diseño de arquitectura bajo el modelo de **Confianza Cero (Zero Trust)**. Aprenderemos cómo llevar este enfoque teórico directamente a la práctica en nuestro código y en la integración de APIs, aplicando sus tres reglas de oro: verificar siempre explícitamente, otorgar únicamente el nivel de acceso de menor privilegio y mantener la postura mental de "asumir la brecha", o *assume breach*, preparando a nuestros sistemas para resistir incluso cuando el perímetro haya sido traspasado.

Para cerrar el bloque y darle una estructura institucional sólida a todo esto, analizaremos el **MSSDL**, el Ciclo de Vida de Desarrollo Seguro de Microsoft. Recorreremos detalladamente sus seis fases obligatorias: desde la definición inicial de **Requerimientos** y **Diseño**, pasando por la **Codificación** y **Verificación**, hasta llegar a las etapas clave de **Lanzamiento** y la **Respuesta** estructurada ante incidentes.

Al terminar este primer bloque, tendrás una visión completa y estructurada sobre cómo se diseña e implementa la gobernanza de seguridad en el software moderno.

Slide 8



En el bloque anterior establecimos las bases de la gobernanza y la cultura, aquí nos vamos a meter de lleno en el terreno táctico. Entraremos al **Ecosistema OWASP** y aprenderemos a construir defensas proactivas para hacer que nuestro software sea resistente por diseño.

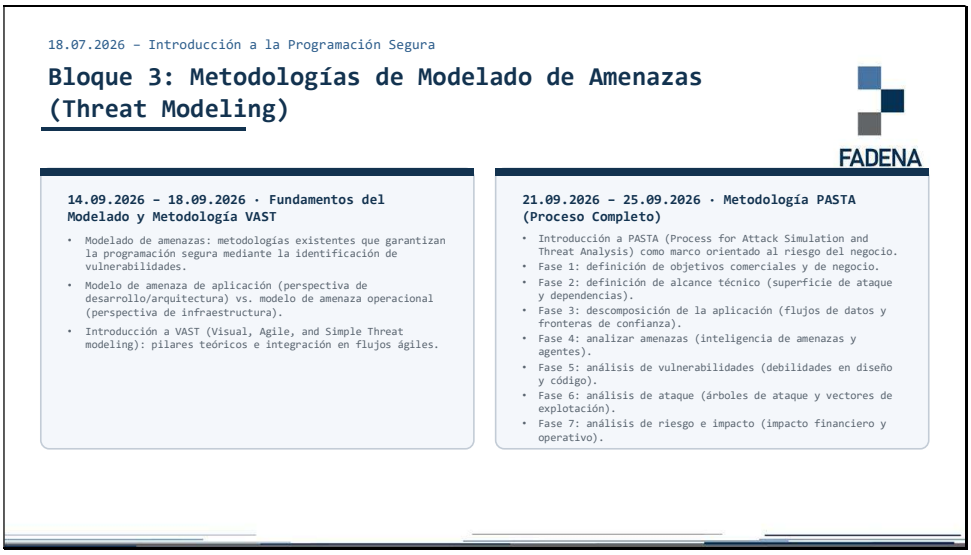
Durante la semana del 7 de Septiembre al 11 de Septiembre daremos inicio a la tercera semana explorando el universo de **OWASP**, la Open Worldwide Application Security Project. OWASP no es solo una lista de vulnerabilidades; es la comunidad colaborativa global más importante en seguridad de software. Analizaremos su filosofía, su estructura y la enorme cantidad de recursos gratuitos, proyectos, estándares y herramientas que ponen a disposición de la industria para combatir la raíz de la inseguridad en el código. Veremos cómo aprovechar estas guías y herramientas para implementar una verdadera gestión de vulnerabilidades, entendiendo cómo se descubren, clasifican y mitigan los fallos más comunes en las aplicaciones modernas.

Y en esa misma semana agregaremos el conocimiento del ecosistema OWASP como respaldo, en la cuarta semana nos enfocaremos en los **Controles Proactivos** y la estrategia de **Defensa en Profundidad**. La idea aquí es simple pero poderosa: no confiamos en una sola capa de seguridad. Aprenderemos la teoría y la aplicación práctica de mecanismos indispensables : la **validación estricta de entradas** para evitar que datos maliciosos entren a nuestro sistema, la **codificación de salidas** para neutralizar ataques como Cross-Site Scripting, y el **manejo seguro de datos sensibles** tanto en tránsito como en reposo.
Para cerrar esta fase, abordaremos tres pilares fundamentales desde la perspectiva del código seguro: la gestión de la identidad, el diseño de esquemas robustos de

autenticación y el uso correcto de la **criptografía** para proteger la confidencialidad e integridad de la información.

Al finalizar este segundo bloque, no solo sabrás dónde buscar recursos de seguridad de clase mundial, sino que tendrás los criterios prácticos para programar defensas proactivas que blinden tus aplicaciones.

Slide 9



En esta etapa nos adentraremos en una de las disciplinas más valiosas y rentables de la ciberseguridad: el **Modelado de Amenazas** o *Threat Modeling*. Aprenderemos a pensar como el atacante para diseñar sistemas intrínsecamente seguros.

En la quinta semana (desde el 14 de Septiembre al 18 de Septiembre) nos enfocaremos en los **Fundamentos del Modelado**. Comprenderemos cómo esta práctica nos permite identificar debilidades en las fases tempranas del desarrollo, lo que reduce radicalmente los costos de corrección. Aprenderemos a diferenciar entre dos perspectivas complementarias: el **Modelo de Amenaza de Aplicación**, centrado en la arquitectura y la lógica de desarrollo, y el **Modelo de Amenaza Operacional**, enfocado en los riesgos de la infraestructura. Además, nos introduciremos en la metodología **VAST** (*Visual, Agile, and Simple Threat modeling*), analizando sus pilares teóricos y descubriendo cómo integrarla sin fricción en equipos que trabajan con metodologías ágiles.

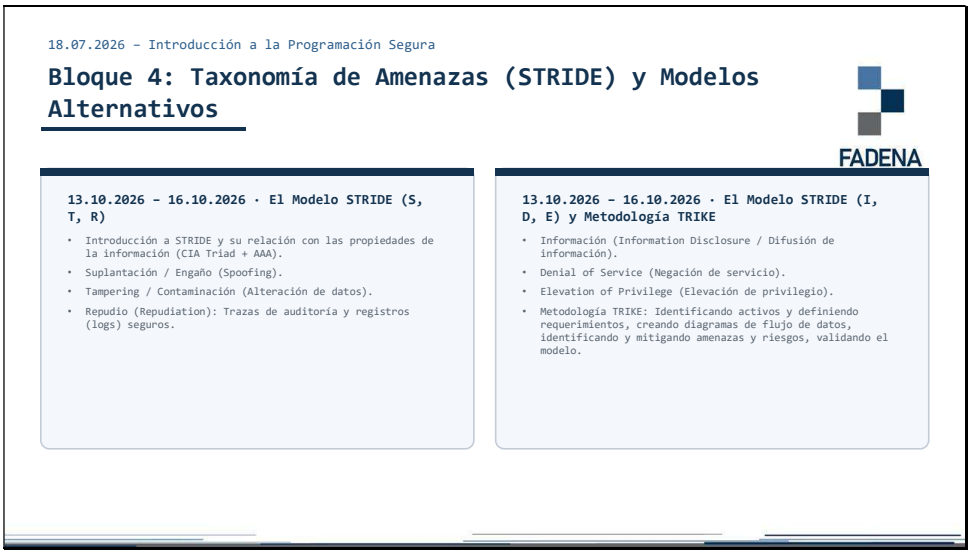
En la sexta semana profundizaremos en una de las metodologías más robustas y completas de la industria: el marco **PASTA** (*Process for Attack Simulation and Threat Analysis*). A diferencia de otros enfoques purely técnicos, PASTA destaca por estar alineado directamente con el riesgo operacional y comercial de la empresa.
Recorreremos detalladamente sus **7 fases**:

- Comenzaremos definiendo los **objetivos de negocio** para entender qué estamos protegiendo realmente.
- Delimitaremos el **alcance técnico**, identificando la superficie de ataque y sus dependencias.
- Pasaremos a la **descomposición de la aplicación**, trazando flujos de datos y fronteras de confianza.

- Analizaremos las **amenazas**, estudiando la inteligencia de amenazas y los posibles agentes atacantes.
- Realizaremos un **análisis de vulnerabilidades** para detectar debilidades.
- Ejecutaremos el **análisis de ataque**, construyendo árboles de ataque y modelando vectores de explotación reales.
- Y finalmente, cerraremos con el **análisis de riesgo e impacto**, cuantificando el verdadero daño financiero y operativo que un ataque podría causar.

Al terminar este bloque, dominarás metodologías para diagnosticar, mapear y neutralizar amenazas en la arquitectura de tu software antes de que lleguen a producción.

Slide 10



En este Bloque vamos a dar un paso más en el nivel de detalle y aprenderemos a clasificar las amenazas de forma sistemática utilizando la taxonomía más famosa de la industria: el modelo **STRIDE**, complementándolo con la metodología alternativa **TRIKE**.

En la semana del 13 de octubre hasta el 16 de octubre comenzaremos introduciendo el modelo **STRIDE** y descubriendo cómo se conecta directamente con las bases de la seguridad de la información: la **Tríada CIA** (*Confidencialidad, Integridad y Disponibilidad*) y el marco **AAA** (*Autenticación, Autorización y Auditoría*).
Analizaremos a fondo las primeras tres letras del mnemónico:

- **S de *Spoofing*** o Suplantación: cómo los atacantes logran engañar al sistema haciéndose pasar por usuarios o servicios legítimos.
- **T de *Tampering*** o Alteración: la contaminación o modificación no autorizada de datos, tanto en tránsito como en bases de datos.
- **R de *Repudiation*** o Repudio: la capacidad de un usuario de negar haber realizado una acción. Aquí aprenderemos a diseñar trazas de auditoría y sistemas de registros (*logs*) seguros que garanticen el no repudio.

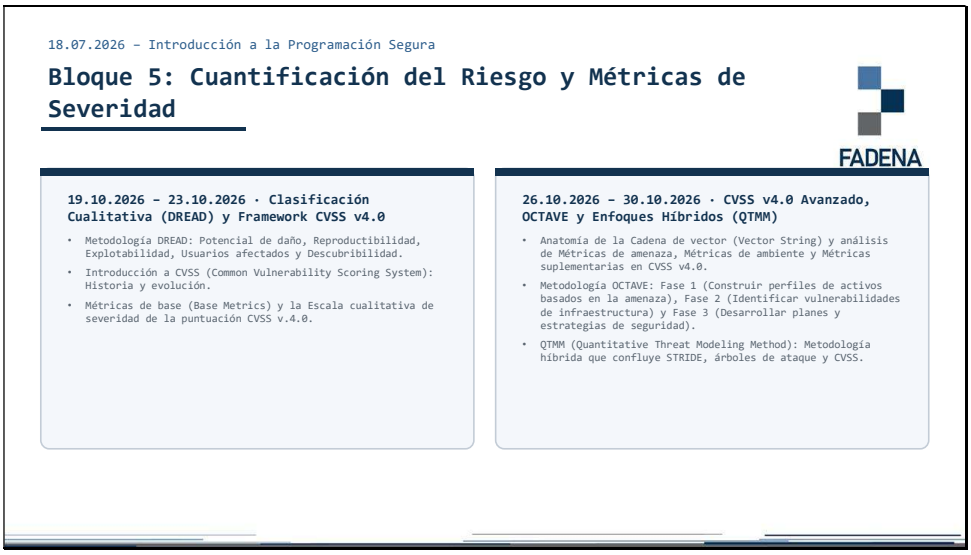
En la misma semana del 13 de octubre hasta el 16 de octubre completaremos la taxonomía abordando las últimas tres letras de STRIDE:

- **I de *Information Disclosure***: la filtración o exposición indebida de datos confidenciales.
- **D de *Denial of Service***: ataques de negación de servicio que atentan contra la disponibilidad del sistema.
- **E de *Elevation of Privilege***: escenarios donde un atacante logra escalar permisos para ejecutar acciones reservadas a administradores.

Para cerrar esta fase y tener una visión aún más amplia, estudiaremos la metodología **TRIKE**. A diferencia de STRIDE, TRIKE adopta un enfoque riguroso centrado en la gestión de activos y en el cumplimiento de requerimientos de seguridad. Aprenderemos a construir diagramas de flujo de datos (*DFDs*), identificar y mitigar riesgos en cada interacción del sistema y, finalmente, validar la efectividad de nuestro modelo.

Luego de este Bloque obtendrás el conocimiento de la taxonomía STRIDE y de la metodología TRIKE, lo que te permitirá catalogar con major precisión vectores de ataque en las aplicaciones.

Slide 11



En los bloques anteriores aprendimos a identificar y categorizar las amenazas en nuestro software. Sin embargo, en el mundo real los recursos son finitos. Entonces, ¿qué vulnerabilidad debemos corregir primero? En este bloque aprenderemos a medir, cuantificar y priorizar el riesgo mediante métricas de severidad estándar en la industria.

Iniciaremos la novena semana desde el 19 de Octubre hasta el 23 de Octubre explorando la metodología **DREAD**, una herramienta de clasificación cualitativa que nos permite evaluar el riesgo a través de cinco dimensiones: el **Potencial de daño**, la **Reproductibilidad** del fallo, su facilidad de **Explotabilidad**, la cantidad de **Usuarios afectados** y su nivel de **Descubribilidad**.
Posteriormente, entraremos de lleno en el estándar global por excelencia: **CVSS** (*Common Vulnerability Scoring System*). Revisaremos su historia y evolución hasta llegar a la versión más reciente, **CVSS v4.0**. Analizaremos sus **Métricas de base** y cómo se traducen en la escala cualitativa de severidad para asignar una nota objetiva a cada fallo descubierto.

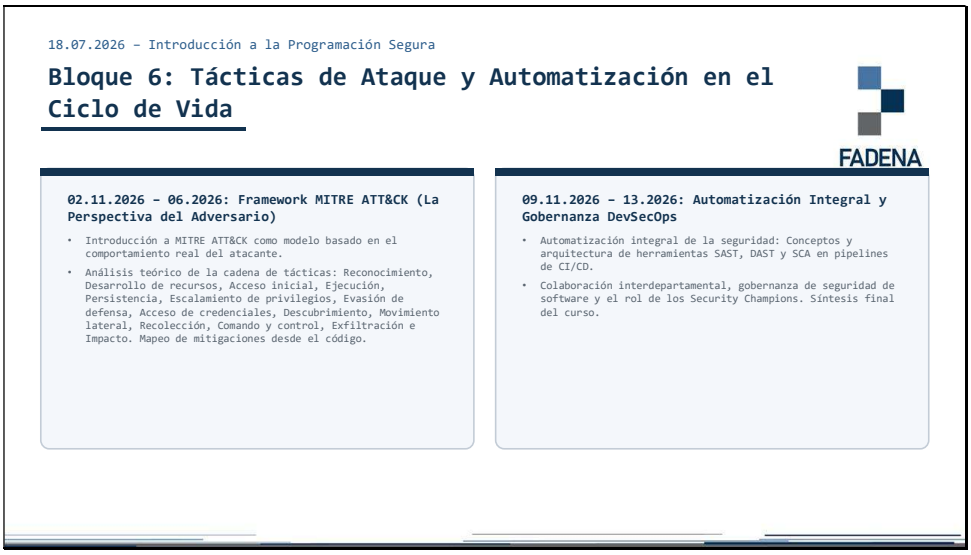
En la décima semana desde el 26 de Octubre hasta el 30 de Octubre llevaremos nuestro dominio de **CVSS v4.0** al siguiente nivel. Aprenderemos a leer y construir la **Cadena de vector** (*Vector String*) y analizaremos las capas que hacen tan precisa a esta versión: las **Métricas de amenaza**, las **Métricas de ambiente** —que ajustan la severidad al contexto real de tu empresa— y las **Métricas suplementarias**.
A continuación, estudiaremos la metodología **OCTAVE**, un enfoque organizativo enfocado en la gestión de activos. Recorreremos sus tres fases clave:

- La construcción de **perfiles de activos** basados en amenazas.
- La identificación de **vulnerabilidades en la infraestructura**.
- Y la elaboración de **planes y estrategias de seguridad** a medida.

Para cerrar este bloque abordaremos **QTMM** (*Quantitative Threat Modeling Method*), una metodología híbrida avanzada que logra confluir la categorización de STRIDE, la profundidad visual de los árboles de ataque y la cuantificación matemática de CVSS.

Al concluir este bloque, tendrás el conocimiento para argumentar con datos sólidos y métricas cuantitativas la severidad de cualquier amenaza, permitiendo a tu equipo priorizar soluciones de forma estratégica.

Slide 12



Este es Nuestro último Bloque. Sérá un viaje completo donde recorreremos la gobernanza, el modelado de amenazas y la cuantificación del Riesgo y en este Bloque es momento de conectar todo lo aprendido con la realidad operativa del día a día: comprender la perspectiva del adversario y llevar la seguridad a la velocidad de la integración continua.

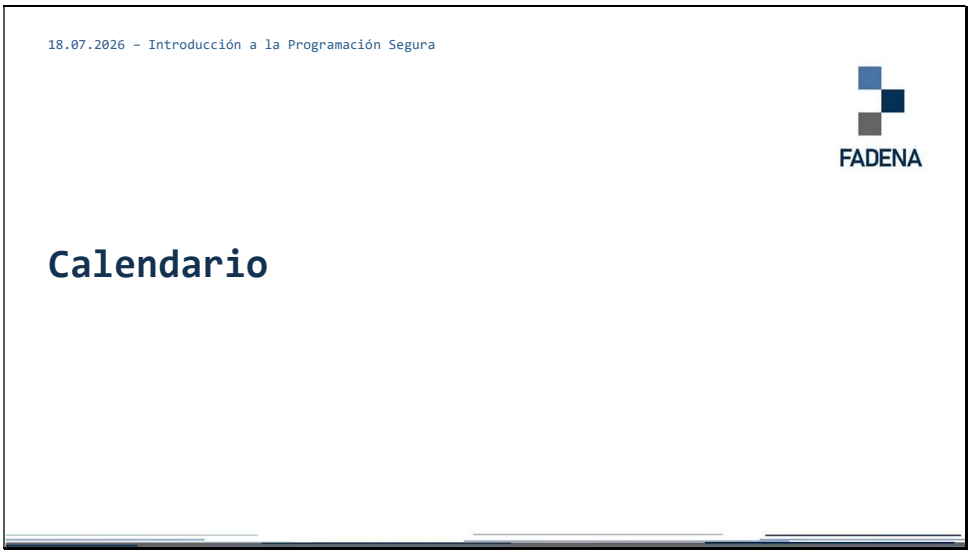
En la primera parte de las semana final nos adentraremos en el **Framework MITRE ATT&CK**, el modelo de referencia global que describe el comportamiento real de los ciberdelincuentes.
Analizaremos minuciosamente toda la cadena de tácticas que utiliza un atacante: comenzando por el **Reconocimiento** y el **Desarrollo de recursos**, siguiendo con el **Acceso inicial**, la **Ejecución**, la **Persistencia**, el **Escalamiento de privilegios** y la **Evasión de defensas**. Luego estudiaremos cómo ejecutan el **Acceso a credenciales**, el **Descubrimiento**, el **Movimiento lateral**, la **Recolección**, el **Comando y control**, la **Exfiltración** y el **Impacto** final sobre el sistema. Y lo más importante: aprenderemos a mapear mitigaciones específicas para interceptar al atacante en cada etapa.

En la segunda parte, completaremos la transformación técnica y cultural hacia DevSecOps mediante la **Automatización Integral**. Aprenderemos a diseñar la arquitectura e integrar herramientas de análisis estático (**SAST**), análisis dinámico (**DAST**) y análisis de composición de software (**SCA**) directamente en las canalizaciones de **CI/CD**, detectando fallos automáticamente antes de cada *despliegue*.
Por último, abordaremos la gobernanza humana y la colaboración interdepartamental. Descubriremos la figura clave del **Security Champion**: desarrolladores dentro de cada equipo que impulsan la cultura de seguridad desde adentro. Haremos una síntesis

integradora de todo el programa para unir la teoría, las metodologías y la automatización en una estrategia sólida de seguridad en el software.

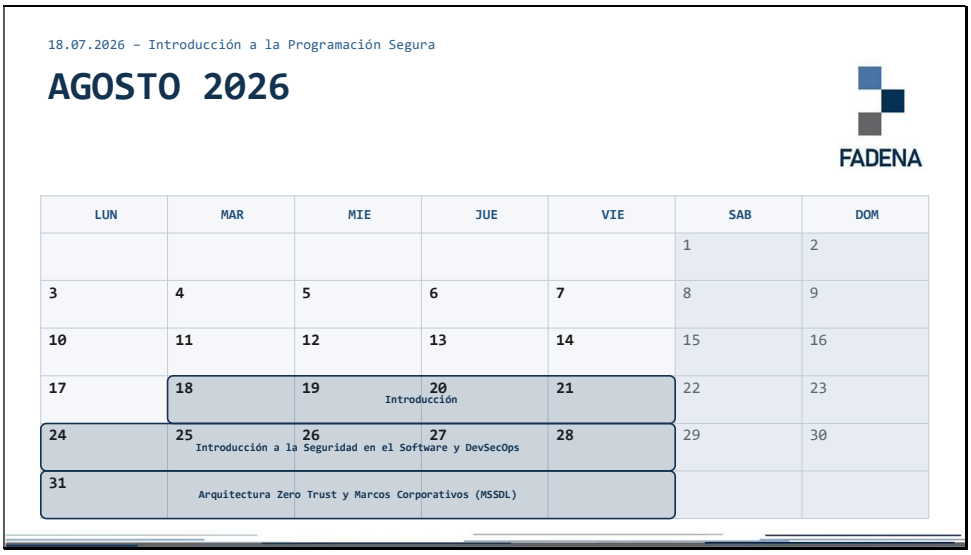
Nuestro objetivo es poder proveerles una visión de 360 grados: sabrás cómo piensa un adversario, cómo automatizar las defensas en el pipeline y cómo liderar una cultura de desarrollo seguro en una organización.

Slide 13



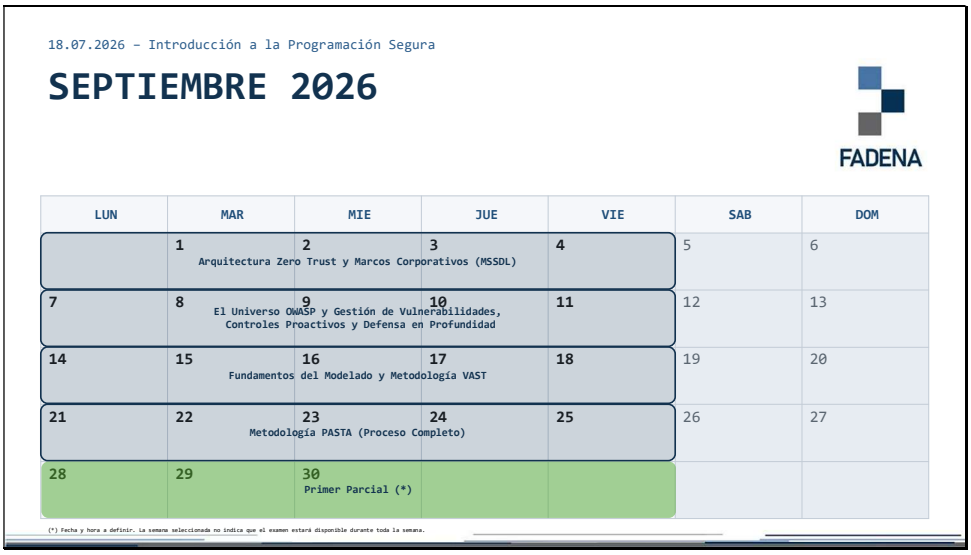
Diapositiva de transición: terminamos el recorrido por el temario y pasamos al calendario académico.
Explicar que a continuación se presenta mes por mes, de agosto a diciembre de 2026, con los temas asignados a cada semana y las instancias de evaluación.
Pedir a los alumnos que registren las fechas de parciales, recuperatorios y examen final, recordando que día y hora se confirman por los canales oficiales de la cátedra.

Slide 14



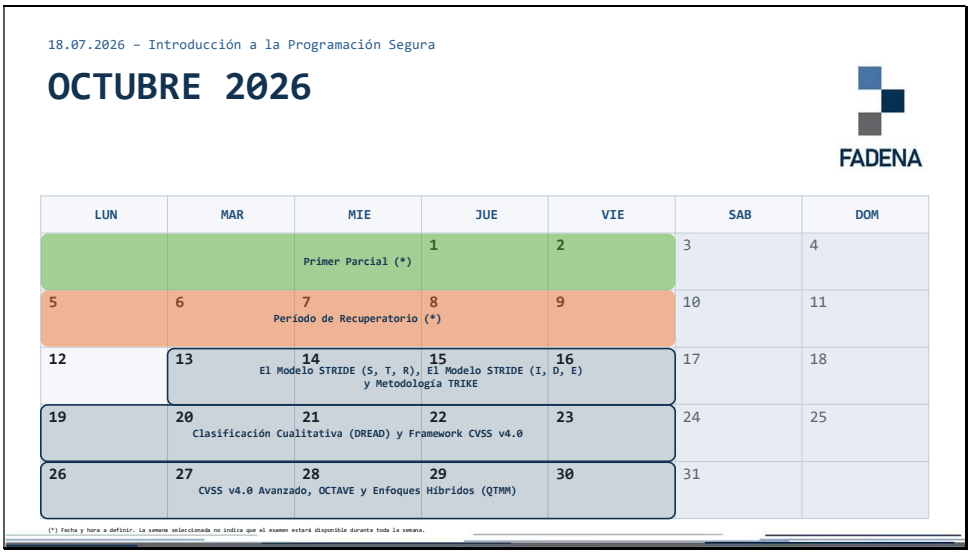
Este calendario resume agosto de 2026 y lo uso para ubicarlos en el cronograma del curso antes de entrar en el contenido técnico.
Les comento que abrimos el mes con el módulo de Introducción, que uso como nivelación: repasamos los conceptos base de seguridad en el software y por qué integro la seguridad desde el diseño y no al final del ciclo.
Después avanzamos a Introducción a la Seguridad en el Software y DevSecOps, donde les explico cómo automatizamos los controles de seguridad dentro del pipeline de CI/CD y qué responsabilidad asume cada rol del equipo.
Cierro el recorrido del mes con Arquitectura Zero Trust y Marcos Corporativos (MSSDL): trabajo el principio de 'nunca confiar, siempre verificar' y muestro cómo el ciclo de desarrollo seguro de Microsoft ordena prácticas, requisitos y puntos de control.
Les recuerdo la referencia del 18.07.2026 - Introducción a la Programación Segura como sesión previa que da contexto a estos temas.
Los invito a marcar las fechas en su agenda y a llegar con el entorno de trabajo preparado para las clases prácticas.

Slide 15



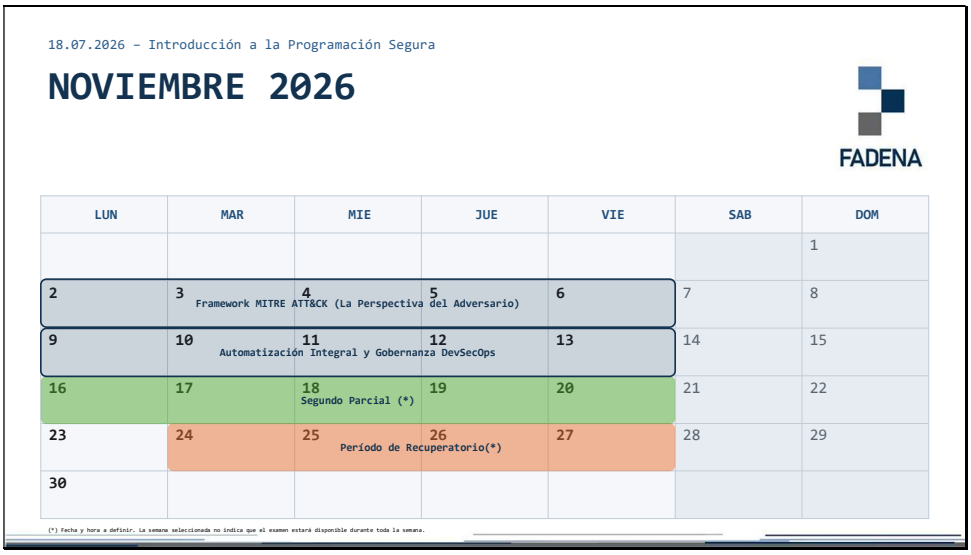
En septiembre continuamos con Arquitectura Zero Trust y Marcos Corporativos (MSSDL) en la primera semana, con lo que cerramos el Bloque 1.
Del 07 al 11 de septiembre dicto el Bloque 2: les presento el Universo OWASP y la gestión de vulnerabilidades, junto con los controles proactivos y la defensa en profundidad.
A partir del 14 comenzamos el Bloque 3 con los fundamentos del modelado y la metodología VAST, y del 21 al 25 desarrollo la metodología PASTA completa.
Les señalo el Primer Parcial marcado en el mes: la semana indicada es orientativa, la fecha y la hora exactas las confirmo por los canales oficiales de la cátedra y el examen no está disponible toda la semana.

Slide 16



En octubre concentro el Bloque 4: trabajamos el modelo STRIDE en dos partes (S, T, R y luego I, D, E) junto con la metodología TRIKE.
Luego avanzamos al Bloque 5, donde les presento la clasificación cualitativa DREAD y el framework CVSS v4.0, y después CVSS v4.0 avanzado, OCTAVE y los enfoques híbridos (QTMM).
Les marco en el calendario el Primer Parcial y el Período de Recuperatorio.
Les repito la aclaración: la semana señalada no implica que el examen esté disponible toda la semana; la fecha y la hora las confirmo desde la cátedra.

Slide 17



En noviembre desarrollamos el Bloque 6: les presento el framework MITRE ATT&CK desde la perspectiva del adversario y mapeamos las tácticas a mitigaciones concretas en el código.
En la segunda parte del mes trabajo la automatización integral y la gobernanza DevSecOps: SAST, DAST y SCA en pipelines de CI/CD, además del rol de los Security Champions.
Les indico el Segundo Parcial y el Período de Recuperatorio previstos para este mes.
Les recuerdo que la fecha y la hora de las evaluaciones las comunico oportunamente; la semana marcada es solo de referencia.

Slide 18



Con diciembre cierro el cronograma: aquí ubicamos el Examen Final.
Aprovecho para sintetizar con ustedes todo el recorrido: partimos de los fundamentos y la gobernanza del desarrollo seguro y llegamos al modelado de amenazas, la cuantificación del riesgo y la automatización de la seguridad.
Les reitero que la fecha y la hora del final las publico por los canales oficiales y que la semana marcada es orientativa.
Cierro abriendo el espacio a consultas y les anticipo la modalidad de estudio y los materiales de la cátedra.